Course Code:CSE2008	Course Title: Operating Systems	TPC	3	2	4
Version No.	1.2				
Course Pre-requisites/ Co-requisites	None				
Anti-requisites (if any).	SWE2007				
Objectives:	1. To study fundamentals of Operating Systems 2. To understand concurrency and control of asynchronous processes, deadlocks, memory management, processor and disk scheduling, parallel processing, and file system organization.				
Expected Outcome:	On completion of the course, students will have the ability to 1. Understand and implement basic services and functionalities of the operating system using system calls. 2. Use modern operating system calls and synchronization libraries in software/ hardware interfaces. 3. Understand the benefits of thread over process and implement synchronized programs using multithreading concepts. 4. Analyze different CPU Scheduling Algorithms. 5. Implement memory management schemes and page replacement schemes. 6. Simulate file allocation and organization techniques. 7. Understand the concepts of deadlock in operating systems and implement them in multiprogramming system.				
Module No. 1	**Introduction**				**5 Hours**
What operating systems do, Computer System Organization, Operating system operations – Process Management, Memory Management, Storage Management, I/O Systems, Protection & Security, Types of Operating Systems – Batch O.S., Multi programmed O.S., Time Sharing O.S., Distributed O.S., Virtualization, Real Time Embedded Systems, Open source O.S., System calls & Overview on Types of System calls.					
Module No. 2	**Process and threads**				**7 Hours**
Process and programs, process states, process concept, process scheduling, operations on processes, IPC, concurrency, interacting processes, multithreading models.					
Module No. 3	**Process Coordination and Deadlock**				**10 Hours**
Process synchronization, critical-section problem, Peterson's solution, synchronization hardware, semaphores, monitors, classic problems of synchronization. System model, deadlock characterization, methods for handling deadlocks, deadlock prevention, deadlock avoidance, deadlock detection, recovery from deadlock.					
Module No. 4	**Memory Management**				**9 Hours**
Memory hierarchy, address spaces, static and dynamic memory, memory allocation to a process, continuous memory allocation, noncontiguous memory allocation, swapping and relocation, paging, segmentation, paging with segmentation.					
Module No. 5	**Virtual Memory and File Management**				**8 Hours**
Virtual memory basics, demand paging, page replacement policies, memory allocation to a process, page faults. File concept, Access methods, File types, File operation, Directory structure, File System structure, Allocation methods (contiguous, linked, indexed), Free-space management (bit vector, linked list, grouping), directory implementation (linear list, hash table), efficiency & performance.					
Module No. 6	**Storage management and security**				**6 Hours**
Secondary-storage structure: disk structure, disk scheduling algorithm, Protection mechanism: protection domain, access control list.					

Text Books

1. Abraham Silberschatz, Peter B. Galvin, Greg Gagne, "Operating System Concepts", Addison-Wesley, 10th edition, 2018.

References

1. Andrew Tanenbaum, "Modern Operating Systems", Prentice Hall, Fourth Edition, 2015.
2. William Stallings, "Operating Systems", Pearson Education, Ninth edition, 2018.

List of Lab Exercises

1 Study of Basic Linux Commands

2 Basic Shell Programs

3 Implementation of System Calls

4 Implementation of Scheduling Algorithms
   A. FCFS Algorithm
   B. CPU Scheduling Using SJF
   C. CPU Scheduling Using Priority
   D. CPU Scheduling Using Round Robin

5 Implementation of Multithreading
   A. Multithreading Using JAVA
   B. Multithreading Using PTHREAD

6 Implementation of Interprocess Communication
   A. IPC Using Semaphore – Producer and Consumer Problem
   B. IPC Using Semaphore – Readers and Writers Problem
   C. IPC Using Semaphore – Dining Philosopher Problem
   D. IPC Using Pipes

7 Implementation of Deadlock Prevention Using Banker's Algorithm

8 Implementation of Memory Management Using Paging

9 Implementation of Memory Management Using Segmentation

10 Implementation of Page Replacement Algorithms
   A. First Come First Serve (FCFS) Page Replacement
   B. Least Recently Used (LRU) Page Replacement Algorithm
   C. Optimal Page Replacement Algorithm

Sample Exercises

1. Write a program to multiply two matrices that uses threads to divide up the work necessary to compute the product of two matrices. There are several ways to improve the performance using threads. You need to divide the product in row dimension among multiple threads in the computation. That is, if you are computing A * B and A is a 10 x 10 matrix and B is a 10 x 10 matrix, your code should use threads to divide up the computation of the 10 rows of the product which is a 10 x 10 matrix. If you were to use 5 threads, then rows 0 and 1 of the product would be computed by thread 0, rows 2 and 3 would be computed by thread 1,...and rows 8 and 9 would be computed by thread 4.
2. Write a C program to implement the following game. The parent program P first creates two pipes, and then spawns two child processes C and D. One of the two pipes is meant for communications between P and C, and the other for communications between P and D. Now, a loop runs as follows. In each iteration (also called round), P first randomly

chooses one of the two flags: MIN and MAX (the choice randomly varies from one iteration to another). Each of the two child processes C and D generates a random positive integer and sends that to P via its pipe. P reads the two integers; let these be c and d. If P has chosen MIN, then the child who sent the smaller of c and d gets one point. If P has chosen MAX, then the sender of the larger of c and d gets one point. If c = d, then this round is ignored. The child process who first obtains ten points wins the game. When the game ends, P sends a user-defined signal to both C and D, and the child processes exit after handling the signal (in order to know who was the winner). After C and D exit, the parent process P exits. During each iteration of the game, P should print appropriate messages (like P's choice of the flag, the integers received from C and D, which child gets the point, the current scores of C and D) in order to let the user know how the game is going on. Name your program childsgame.c	
Mode of Evaluation	Cumulative Lab Assessment 25% Continuous Assessment Test-1 20% Continuous Assessment Test-2 20% Final Assessment Test 20% Digital Assignment /Mini Project 15%
Modified by	Dr. Saroj Kumar Panigrahy and Dr. Aravapalli Rama Satish
Recommended by the Board of Studies on	07.01.2021
Date of Approval by the Academic Council	5th AC (02.03.2021)